

Pre-requisites

1. IBM Cloud Account Setup: [IBM Cloud Console](#)
2. Watsonx Access and Configuration: [Watsonx Overview](#)
3. Python Basics: [Python.org](#)
4. FastAPI Framework: [FastAPI Docs](#)

Project Workflow

1. Gemini AI Setup & Initialization :

- **Activity 1.1:** Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2:** Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3:** Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development :

- **Activity 2.1:** Design Flask-based backend structure for routing, session control, and planner modules.
- **Activity 2.2:** Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- **Activity 2.3:** Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py) :

- **Activity 3.1:** Define Flask routes like [/generate-home](#), [/generate-party](#), [/generate-jewelry](#).
- **Activity 3.2:** Organize backend using modular structure:
 - [routes/](#): Endpoints
 - [services/](#): Recommendation logic & AI calls
 - [models/](#): Input/output schemas

- **Activity 3.3:** Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4:** Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development :

- **Activity 4.1:** Build responsive HTML forms for:
 - Home Interior Planner (room details, quantities)
 - Party Planner (event type, budget, guests)
 - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2:** Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization :

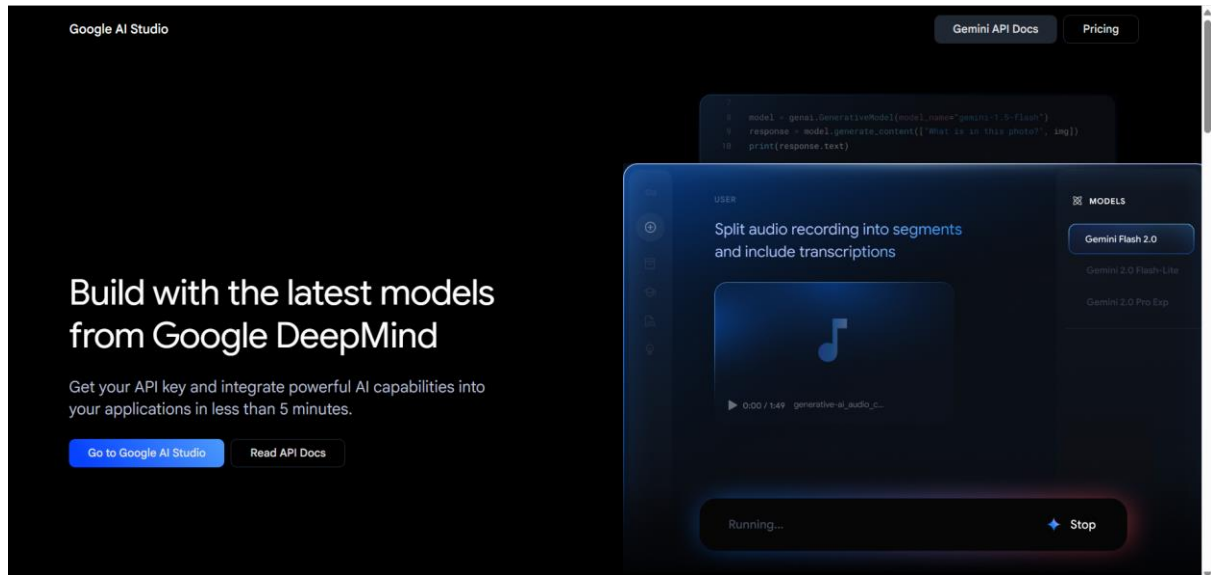
- **Activity 5.1:** Perform testing with real-world use cases across all three planners.
- **Activity 5.2:** Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3:** Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4:** Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization :

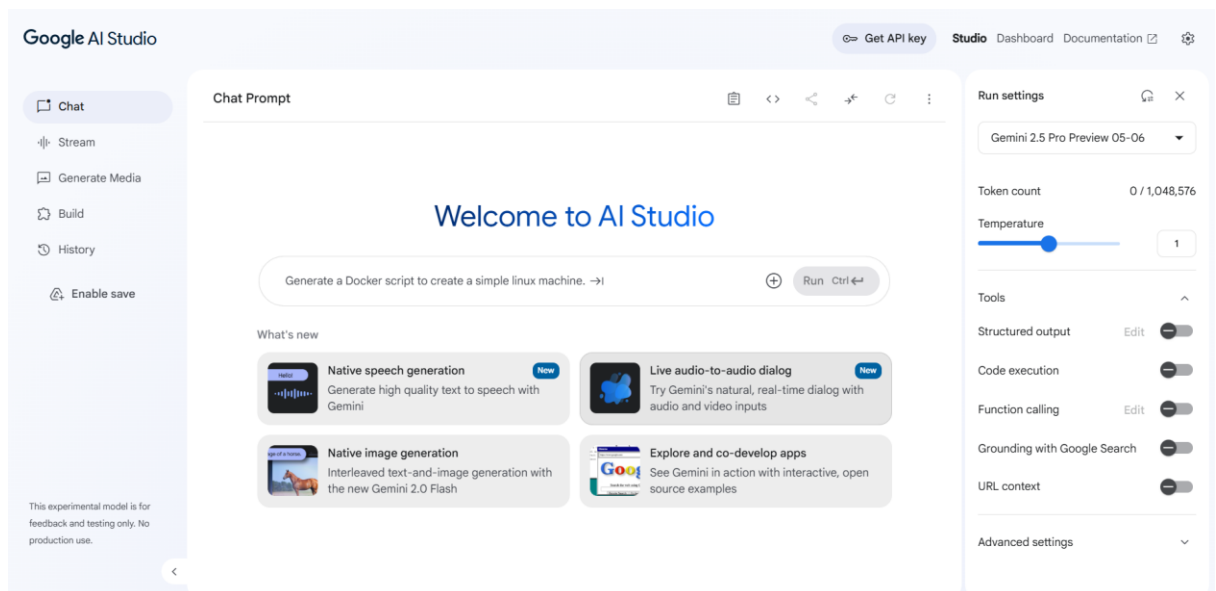
- **Objective:** Establish and configure the core AI infrastructure for PocketSmart AI.
- **Core Model:** Relies on Gemini 1.5 Flash Pro to interpret user inputs, process budgets, and generate personalized recommendations.
- **Key Capabilities:** Equips the development environment with authenticated access to Gemini's APIs for text-based reasoning and image analysis (specifically for the Jewelry Planner).

Activity 1.1: Set up Google Cloud account and enable Gemini API

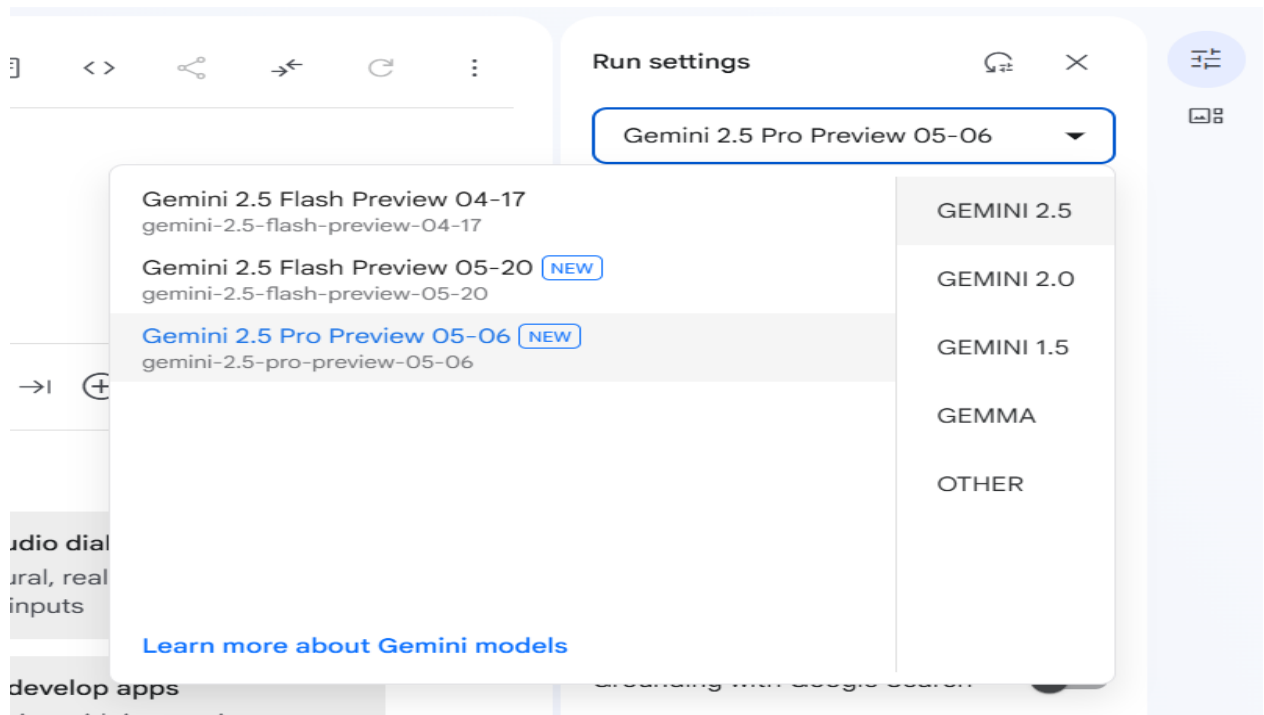
- Sign in or create a Google Cloud account via console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

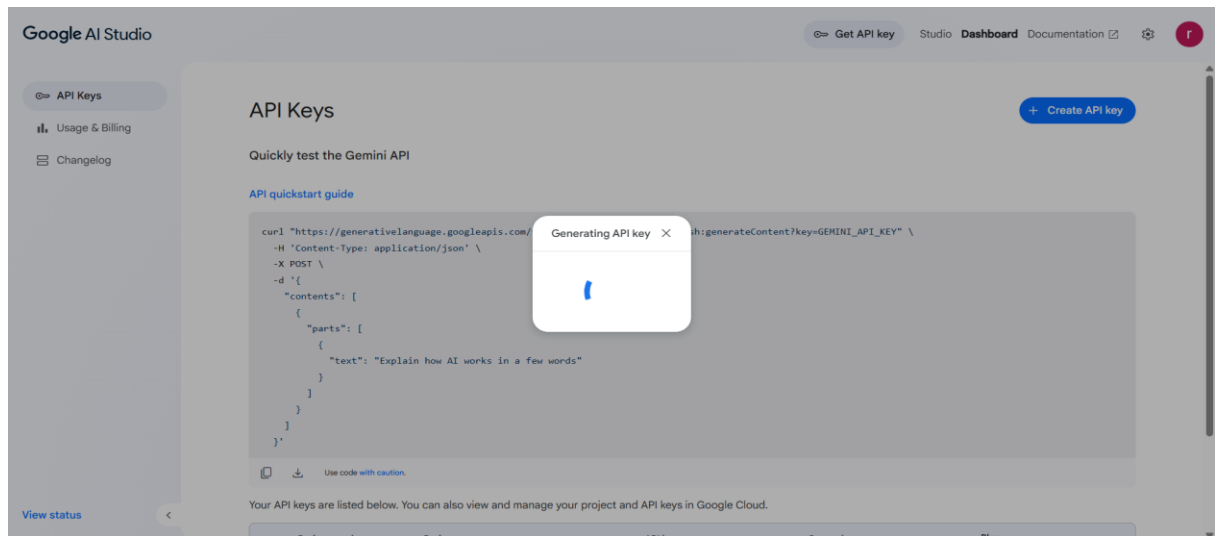


Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key



- Once click on API key you will the api key and make sure to copy the API key



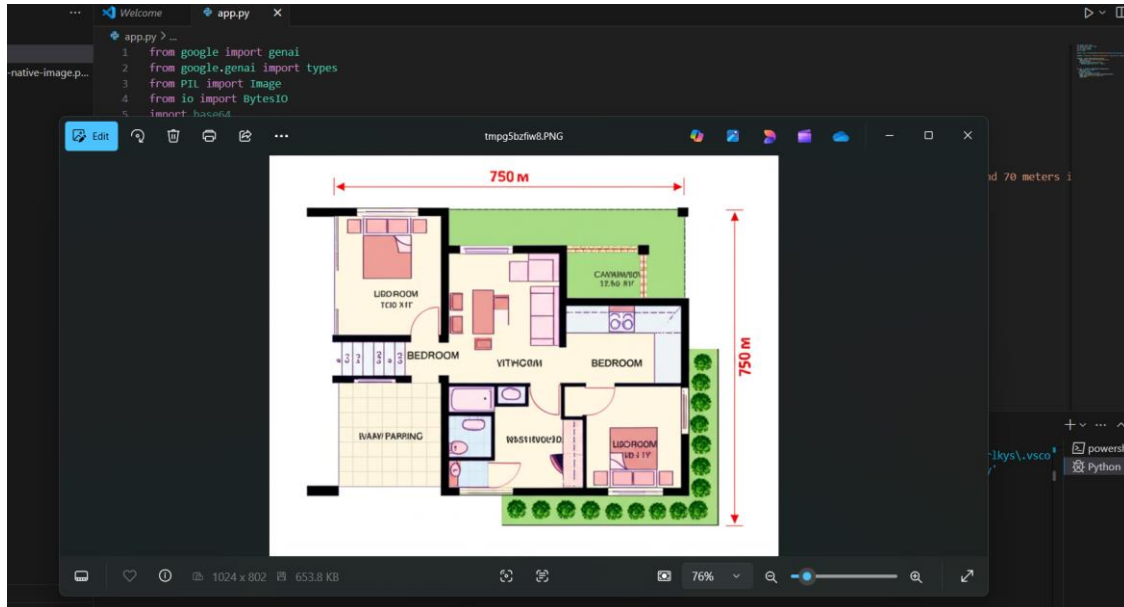
Activity 1.3: Validate Gemini API connectivity

- Make a sample prompt call using Python to confirm integration.

```

app.py > ...
1 from google import genai
2 from google.genai import types
3 from PIL import Image
4 from io import BytesIO
5 import base64
6
7 client = genai.Client(api_key="AIzaSyDdzTxAIisLXlQ")
8
9 contents = ('Generate a detailed 2D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters
10
11 response = client.models.generate_content(
12     model="gemini-2.0-flash-exp-image-generation",
13     contents=contents,
14     config=types.GenerateContentConfig(
15         response_modalities=['Text', 'Image']
16     )
17 )
18
19 for part in response.candidates[0].content.parts:
20     if part.text is not None:
21         print(part.text)
22     elif part.inline_data is not None:
23         image = Image.open(BytesIO((part.inline_data.data)))
24         image.save('gemini-native-image.png')
25         image.show()
  
```

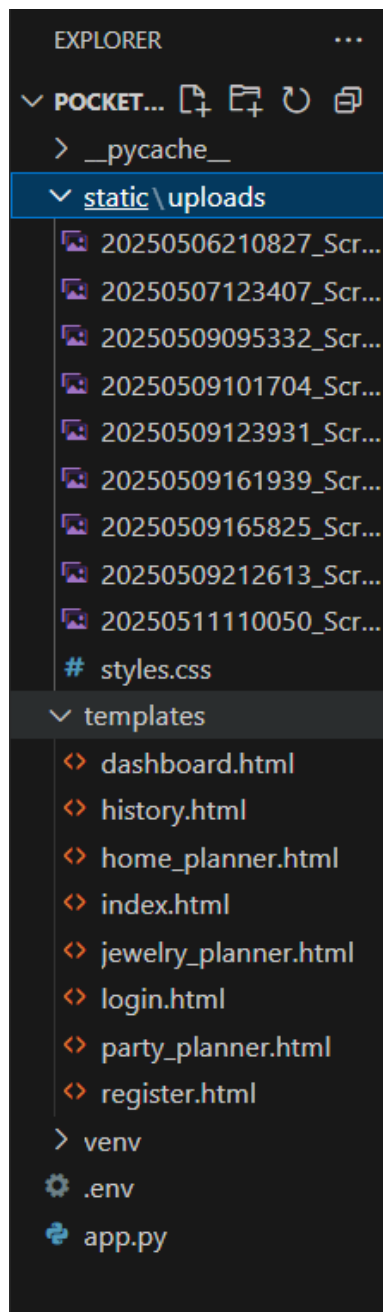
- Test both text-only and image + text prompts to ensure multimodal compatibility.



MILESTONE 2: Core Functionalities Development :

- **Framework & Model:** Built on Flask leveraging Google's Gemini 1.5 Flash Pro foundation model.
- **Core Inputs:** Accepts user budgets and optional multimodal image inputs.
- **Domain Segmentation:** Uses modular planning logic tailored for three specific categories: Home, Party, and Jewelry.
- **Centralized Logic:** Abstracted into a utility file (gemini_utils.py) to manage prompt orchestration, budget formatting, and image analysis.
- **System Resilience:** Implements fallback strategies to maintain recommendations when product matches are limited.

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

```
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

- Fast API Initialization

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

- Load .env environment

```
# Load environment variables
load_dotenv()
# Configure Google Gemini AI - Do this once, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # Try alternative environment variable name
    API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("No Google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False) # Allow optional token

class RegisterUser(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

- **/generate-home**: Generates home interior recommendations (furniture, decor, lighting)

```
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """Generate home interior recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need interior design product recommendations for a home in India with a total budget of ₹{budget_input.total_budget:.2f}.
        Requirements:
        - {budget_input.num_lights} lights/lighting fixtures
        - {budget_input.num_fans} ceiling fans
        - {budget_input.num_furniture} furniture pieces
        - {budget_input.num_dining_tables} dining tables

        Additional rooms to consider:
        {("- Living room" if budget_input.has_living_room else "")}
        {("- Kitchen" if budget_input.has_kitchen else "")}
        {("- Bedroom" if budget_input.has_bedroom else "")}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with product recommendations **available in India**.
        Use **Indian brands and pricing**. Include **search terms** suitable for Indian shopping platforms.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "lighting",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
            ]
        }}
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """
    """
    }
    })
    },
    "calculation_table": [
        {
            "category": "",
            "items_count": 0,
            "total_cost": 0.0,
            "percentage_of_budget": 0.0
        }
    ],
    "remaining_budget": 0.0,
    "additional_suggestions": []
}

Ensure total costs stay within budget. Include search terms for each item to find on shopping websites like Flipkart, Amazon India, IKEA I
"""

response = model.generate_content(prompt)
result = extract_json_from_response(response.text)

# Add shopping links for each item
for category in result.get("budget_breakdown", []):
    for item in category.get("items", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "ikea": f"https://www.ikea.com/in/en/search?q={urllib.parse.quote_plus(search_terms)}",
                "mynta": f"https://www.mynta.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "ajio": f"https://www.ajio.com/search/?text={urllib.parse.quote_plus(search_terms)}"
            }

    return result
```

```
except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

- **/generate-party**: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
            ]
        }}
        """
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    },
    "venue_suggestions": [
        {{
            "name": "",
            "type": "",
            "capacity": 0,
            "estimated_cost": 0.0,
            "search_terms": ""
        }}
    ],
    "remaining_budget": 0.0,
    "additional_suggestions": []
}}

Ensure all costs are in INR and total does not exceed the given budget.
Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart, etc.
"""

response = model.generate_content(prompt)
result = extract_json_from_response(response.text)

# Create INR calculation table
result["calculation_table_inr"] = []
categories = {}

for category in result.get("budget_breakdown", []):
    cat_name = category.get("category", "Misc")
    cat_allocation = category.get("allocation", 0)

    if cat_name not in categories:
        categories[cat_name] = {
            "category": cat_name,
            "items_count": 0,
            "total_cost": 0,
            "percentage_of_budget": 0
        }
```

- linking the websites like amazon,flipkart,ikea ect.... to get the products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for item in category.get("items", []):
        categories[cat_name]["items_count"] += 1
        categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

    if result["total_budget"] > 0:
        categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

    for cat_data in categories.values():
        result["calculation_table_inn"].append(cat_data)

    # Define relevant shopping platforms for each category
    category_platforms = {
        "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
        "catering": ["swiggy", "zomato"],
        "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
        "decoration": ["amazon", "flipkart", "meesho", "myntra"],
        "entertainment": ["bookmyshow", "amazon", "flipkart"],
        "gifts": ["amazon", "flipkart", "myntra", "meesho"],
        "photography": ["google", "amazon", "flipkart"],
        "music": ["amazon", "flipkart", "bookmyshow"],
        "games": ["amazon", "flipkart"],
        "accessories": ["amazon", "flipkart", "myntra", "meesho"],
        "transportation": ["makemytrip", "google"],
        "return_gifts": ["amazon", "flipkart", "myntra", "meesho"]
    }

    # Default platforms if category is not in our predefined list
    default_platforms = ["amazon", "flipkart", "google"]

    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
```

- using the if condition to categorize the situation based products

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {}

                # Only add relevant platforms for this category
                if "amazon" in relevant_platforms:
                    item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                if "flipkart" in relevant_platforms:
                    item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bigbasket" in relevant_platforms:
                    item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                if "swiggy" in relevant_platforms:
                    item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                if "zomato" in relevant_platforms:
                    item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "bookmyshow" in relevant_platforms:
                    item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "myntra" in relevant_platforms:
                    item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "meesho" in relevant_platforms:
                    item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "google" in relevant_platforms:
                    item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                if "booking" in relevant_platforms:
                    item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                if "makemytrip" in relevant_platforms:
                    item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
                if "oyorooms" in relevant_platforms:
                    item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                if "nobroker" in relevant_platforms:
                    item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"
```

```
# Add search links for venue suggestions
venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
for venue in result.get("venue_suggestions", []):
    search_terms = venue.get("search_terms", "")
    if search_terms:
        venue["search_links"] = {}
        if "google" in venue_platforms:
            venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
        if "booking" in venue_platforms:
            venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
        if "makemytrip" in venue_platforms:
            venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(search_terms)}"
        if "oyorooms" in venue_platforms:
            venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
        if "nobroker" in venue_platforms:
            venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms)}"

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

- **/generate-jewelry**: Accepts text + image inputs to return style-matched jewelry options.

```
● async def jewelry_planner(request: Request, current_user:
    UserInDB = Depends(get_current_active_user)):
    """Jewelry budget planner page"""
```

```
● return templates.TemplateResponse("jewelry_planner.html",  
    {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:  
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""  
    try:  
        # Base prompt with INR context  
        base_prompt = f"""  
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.  
  
        Occasion: {budget_input.occasion}  
        Preferences: {budget_input.preferences or "Not specified"}  
        Provide only India-relevant styles, availability, and price ranges in INR.  
        """  
  
        if image_path:  
            # Include image-based outfit analysis  
            img = Image.open(image_path)  
            prompt = base_prompt + """  
  
            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.  
  
            Format the output as JSON:  
            {  
                "outfit_analysis": {  
                    "colors": [],  
                    "style": "",  
                    "formality": ""  
                },  
                "total_budget": 0.0,  
                "jewelry_recommendations": [  
                    {  
                        "item_type": "",  
                        "description": "",  
                        "style": "",  
                        "estimated_price": 0.0,  
                        "search_terms": ""  
                    }  
                ],  
                "remaining_budget": 0.0,  
            }  
            """
```

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    "styling_tips": []
    }

    Make sure prices are in INR and stay within budget.
    Include Indian-friendly search terms for shopping.
    """
    response = model.generate_content([prompt, img])
else:
    # Only text input
    prompt = base_prompt + """

    Format the output as JSON:
    {
        "total_budget": 0.0,
        "jewelry_recommendations": [
            {
                "item_type": "",
                "description": "",
                "style": "",
                "estimated_price": 0.0,
                "search_terms": ""
            }
        ],
        "remaining_budget": 0.0,
        "styling_tips": []
    }

    Keep prices in INR and relevant to Indian brands.
    """
    response = model.generate_content(prompt)

result = extract_json_from_response(response.text)

# Add shopping links for each item (India-specific)
for item in result.get("jewelry_recommendations", []):
    search_terms = item.get("search terms", "")
```

- linking the websites like amazon,flipkart,ikea ect.... to get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
            }

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

- Logout Page

/logout: Terminates the current user session and redirects to the login page.

```
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTErrors:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")
```

Activity 2.4: Ensure modular code structure

- **/token:** Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token, # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response
```

- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60, # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

MILESTONE 3: Backend – FastAPI Integration (**main.py**) :

- **AI Bridge**: Connects the frontend interface to a Gemini-based AI engine.
- **Modular Design**: Provides API endpoints for Home, Party, and jewelry planners.
- **Key Features**: Includes CORS configuration, session handling, and structured input/output models for data reliability.

❖ **Activity 3.1: Define FastAPI Routes**

- **Home Planner (/generate-home)**: Accepts home-related preferences and budget to return tailored product suggestions.

```
@app.post("/home-budget")
✓ async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    ✓ if current_user.username in active_sessions:
    ✓     active_sessions[current_user.username].user_data["last_home_budget"] = {
        "timestamp": datetime.utcnow().isoformat(),
        "budget": budget_input.total_budget,
    ✓     "requirements": {
        "lights": budget_input.num_lights,
        "fans": budget_input.num_fans,
        "furniture": budget_input.num_furniture,
        "dining_tables": budget_input.num_dining_tables
    }
    }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    ✓ save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

```
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.datetime.now().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```
# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.datetime.now().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result
@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
```

```
    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

- User Auth ([/register](#), [/login](#), [/logout](#), [/token](#)): Manages user registration, authentication, and session control.
- Session Info ([/session-info](#), [/session-data](#)): Retrieves user-specific session data for personalized AI interaction.

❖ Activity 3.2: Modular Architecture Setup

[/recommendations-details](#): Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```
# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")
```

❖ Activity 3.3: CORS & Static Routing

`/history`: Retrieves the user's past recommendation queries and results for review or re-use.

```
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

❖ Activity 3.4: Startup and main function

`/startup`: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800 # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when the script is run directly.

```
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

MILESTONE 4: UI Development :

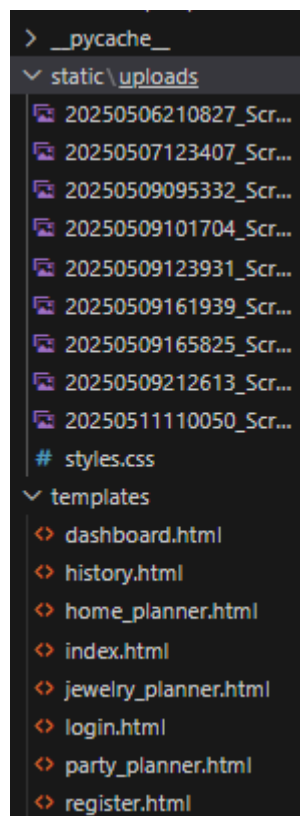
- **Core Goal:** Create a fast, user-friendly, and responsive frontend so users can interact with AI-driven budget recommendations.
- **Supported Categories:** Home Interior, Party Planning, and Jewelry.

- **Backend Integration:** Links directly with a FastAPI backend to take user inputs in real time and display structured AI responses.
- **Key Features:** User authentication, personalized session management, multi-category input, and clear recommendation displays.

❖ Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



MILESTONE 5: Testing & Optimization :

- Goal: Ensure PocketSmart AI's recommendation engine is accurate, reliable, and context-aware.

- Optimization: Fine-tuning AI prompts, improving session handling, and ensuring smooth UI/UX across all devices.

❖ **Activity 5.1:** Test with Real-World Budget Inputs Across Domains

- Conduct end-to-end testing using varied budgets and user preferences for:
 - Home decor setups from platforms like IKEA, Amazon
 - Party packages including food (Swiggy/Zomato), venues (OYO), and decor
 - Jewelry suggestions matched with outfit styles, occasions, and image inputs

✓ **Home Page**

Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.

PocketSmart

AI-Powered Budget Planning for Everyday Needs

Make smarter financial decisions with personalized budget recommendations for home interiors, parties, and jewelry purchases. Our AI helps you get the most value for your money.

[Get Started](#)[Learn More](#)

Our Smart Budget Planners

Discover how PocketSmart helps you make better financial decisions across different areas of your life



Home Interior Budget Planner

Get personalized recommendations for furniture, lighting, and decor that fit your style preferences and budget constraints. Our AI helps you create a beautiful space without overspending.



Party Budget Planner

Plan your perfect event with smart budget allocations for venue, catering, decorations, and entertainment. Our AI suggests the best ways to create memorable events while staying within your budget.



Jewelry Budget Planner

Find the ideal jewelry pieces for any occasion that match your outfit and budget. Our AI recommends options based on your style preferences, occasion, and available budget.

✓ Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform's effectiveness.

What Our Users Say

Real experiences from people who have transformed their financial planning with PocketSmart

"PocketSmart helped me furnish my new apartment without breaking the bank. The recommendations were spot on and I saved nearly 30% of my original budget!"

 **Sarah K.**
Home Owner

"Planning my daughter's birthday party was so much easier with PocketSmart's budget breakdown. The AI suggestions for affordable decorations and catering options were fantastic."

 **Michael R.**
Parent

"The jewelry recommendations perfectly matched my outfit for the wedding. Saved me hours of searching and I received so many compliments on my accessories!"

 **Priya M.**
Fashion Enthusiast

Ready to Optimize Your Budget?

Join PocketSmart today and start making smarter financial decisions across all areas of your life.

[Sign In](#)[Create Account](#)

✓ Footer Part

✓ Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.

PocketSmart

AI-Powered Budget Planning

Create Your Account

Username

 Choose a username

Email

 Enter your email

Password

 Create a strong password

Confirm Password

 Confirm your password

Create Account 

Already have an account? [Sign In](#)

✓ **Login Page**

Login Page: Authenticates existing users to access their dashboard and saved data.

PocketSmart

AI-Powered Budget Planning

Welcome Back

Username

 Enter your username

Password

 Enter your password

[Forgot Password?](#)

Sign In →

Don't have an account? [Create Account](#)

✓ **User Dashboard**

User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.

 PocketSmart

[Home Planner](#) [Party Planner](#) [Jewelry Planner](#) [History](#) [Logout](#)

Welcome, sai!

Choose a budget planner to get started with your personalized financial planning experience



Home Budget Planner

Plan your interior design budget efficiently with AI-powered recommendations for furniture, lighting, and more.

[Get Started](#)



Party Budget Planner

Plan your perfect event with budget allocations for venue, catering, decorations, and entertainment.

[Get Started](#)



Jewelry Budget Planner

Find the ideal jewelry pieces for any occasion that match your outfit and stay within your budget.

[Get Started](#)

[View All Recommendation History](#)

 Recent Activity

**Home Budget Plan**
Budget: \$5,000 - Created on May 7, 2025

**Birthday Party Budget Plan**
Budget: \$1,200 for 30 guests - Created on May 5, 2025

**Jewelry Budget Plan**
Budget: \$800 for Anniversary - Created on April 29, 2025

© 2025 PocketSmart. All rights reserved.

Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.

Home Interior Budget Planner

Create a customized budget plan for your dream home interior

Budget Details

Total Budget (\$)

5000

Fixtures & Furniture

Number of Lights/Fixtures

5

Number of Ceiling Fans

4

Number of Furniture Pieces

2

Number of Dining Tables

1

Rooms to Include

☒ Living Room

☒ Kitchen

☐ Bedroom

Additional Information

Special Requirements or Preferences

Any specific requirements or preferences...

Generate Recommendations

Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.

 Generate Recommendations

Your Personalized Budget Plan

Budget Summary

Total Budget: \$5000.00

Remaining Budget: \$500.00

Lighting

Allocation: \$1500.00

Item	Description	Price	Quantity	Shopping Links
LED Bulb (Warm White)	Energy-efficient LED bulbs for general lighting.	\$100.00	5	Amazon Flipkart Ikea Myntra Ajio

Ceiling_fans

Allocation: \$2000.00

Item	Description	Price	Quantity	Shopping Links
Havells Ceiling Fan	Basic, functional ceiling fan.	\$500.00	4	Amazon Flipkart Ikea Myntra Ajio

Furniture

Allocation: \$1000.00

Item	Description	Price	Quantity	Shopping Links
Plastic Chair	Stackable plastic chairs for kitchen or living room.	\$250.00	2	Amazon Flipkart Ikea Myntra Ajio
Small Wooden Table	Simple wooden table for dining or side table.	\$500.00	1	Amazon Flipkart Ikea Myntra Ajio

Additional Suggestions

- ✓ Consider purchasing used furniture for further cost savings.
- ✓ Look for sales and discounts on online marketplaces.
- ✓ Prioritize essential items and postpone non-essential purchases.

Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count

PocketSmart

Home PlannerParty PlannerJewelry PlannerHistoryLogout

Party Budget Planner

Plan your perfect event with AI-powered budget recommendations

\$ Basic Information

Total Budget (₹)

₹ 5000

Number of Guests

3

Event Details

Party Type

Wedding

Venue Type

Home

Party Needs

Catering

Decoration

Entertainment

Additional Requirements

Special requests, themes, dietary restrictions, etc.

Generate Budget Plan

© 2025 PocketSmart. All rights reserved.



Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.

Your Party Budget Plan

[Print/Save](#)

Budget: ₹5000.00

Venue ₹0.00

Home ₹0.00

Utilizing the home as the venue.

Shop on: [Google](#) [Booking](#) [MakeMyTrip](#) [Oyaraams](#) [Nobroker](#)

Catering ₹2000.00

Home-cooked meal ₹2000.00

Simple, home-cooked meal for 3 people.

Shop on: [Swiggy](#) [Zomato](#)

Entertainment ₹2000.00

Streaming service subscription ₹500.00

One-time purchase of a movie or renting a few movies on a platform like Netflix, Amazon Prime Video, or Hotstar.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Board games/cards ₹500.00

Purchase of a new board game or card game.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Music playlist ₹0.00

Creating a celebratory playlist on Spotify or YouTube Music.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Small gift for couple ₹1000.00

A small token of appreciation for the couple.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Contingency ₹1000.00

Unexpected expenses ₹1000.00

Buffer for unforeseen costs.

Shop on: [Amazon](#) [Flipkart](#) [Google](#)

Total Budget ₹5000.00

Allocated ₹5000.00

Remaining ₹0.00

Venue Suggestions

Home ₹0.00

Type: Residential

Location: undefined

undefined

[Website](#) [View on Map](#)

Additional Suggestions

- Consider making the meal a potluck style if comfortable with guests to reduce catering costs.
- Look for discounts or offers on streaming services or board games.
- Homemade decorations can be a cost effective alternative if you decide to add some.

Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.

PocketSmart

DashboardHome BudgetParty BudgetJewelry Budget

Logout

Jewelry Budget Planner

Get AI-powered jewelry recommendations within your budget for any occasion

Budget Details

Total Budget (\$)

5000

Occasion & Preferences

Occasion

Birthday

Style Preferences

Describe your style preferences, materials, colors, etc.

Upload Outfit Image

Remove Image

Get Recommendations

© 2025 PocketSmart | Smart Budget Solutions

Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.

Get Recommendations

Your Personalized Jewelry Recommendations

Budget Summary

Total Budget: \$5000.00

Remaining Budget: \$800.00

Outfit Analysis

Colors: blue, white

Style: casual

Formality: informal

Jewelry Recommendations

bracelet

Description: A simple, braided leather bracelet with metal accents. This complements the casual style of the outfit without being overly flashy.

\$500.00

Style: casual

Shop For This:

Amazon

Flipkart

Bluestone

Tanishq

CaratLane

Meloma

Meesha

ring

Description: A silver or dark grey metal ring with a minimalist design. Avoid anything too large or ostentatious to maintain the casual feel.

\$700.00

Style: minimalist

Shop For This:

Amazon

Flipkart

Bluestone

Tanishq

CaratLane

Meloma

Meesha

watch

Description: A classic, simple watch with a leather or metal band. A darker band would complement the shirt's colors.

\$3000.00

Style: classic

Shop For This:

Amazon

Flipkart

Bluestone

Tanishq

CaratLane

Meloma

Meesha

Styling Tips

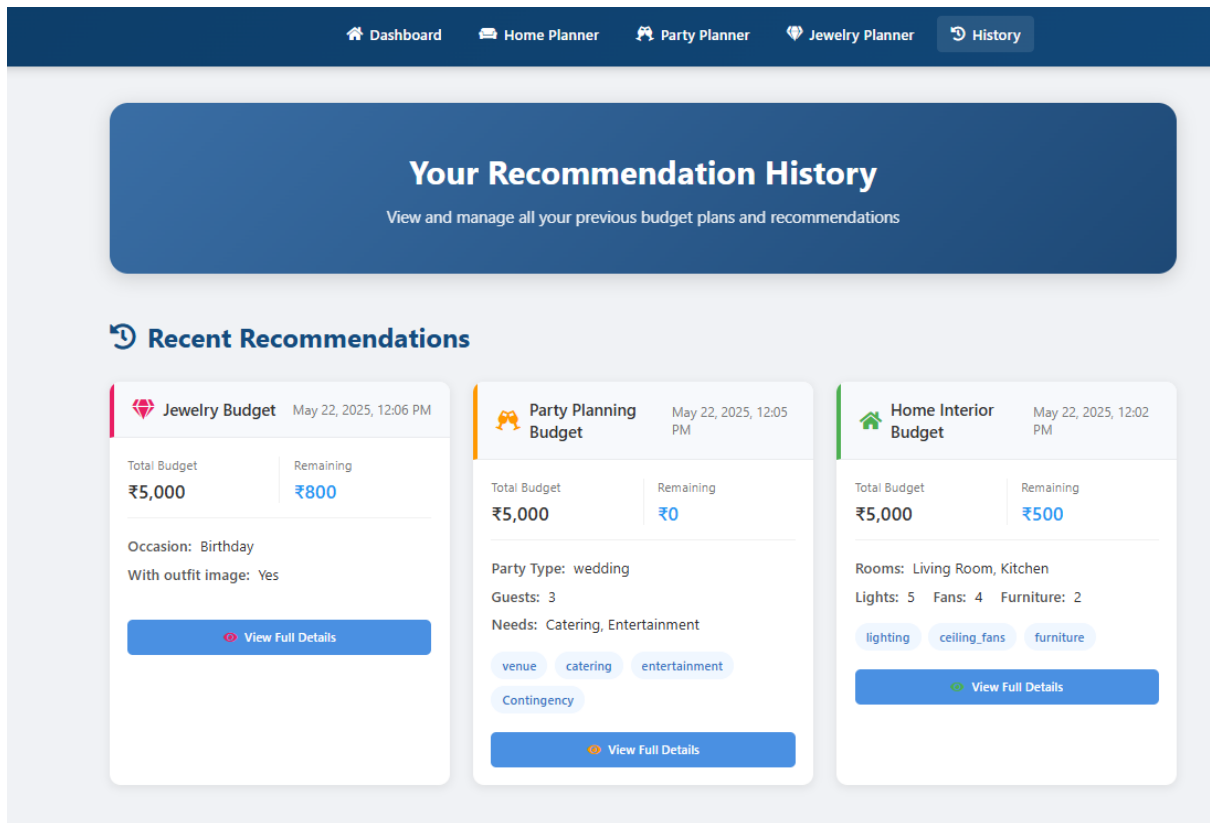
Keep the jewelry minimal to match the casual style of the outfit.

Consider the watch as a statement piece, choosing a design that reflects personal style.

Ensure the metal tones of the ring and bracelet (if metal accents are chosen) complement each other.

© 2025 PocketSense | Smart Budget Solutions

Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.



The screenshot shows the 'Your Recommendation History' page. At the top, a dark blue navigation bar contains links for Dashboard, Home Planner, Party Planner, Jewelry Planner, and History. Below this, a large blue header reads 'Your Recommendation History' with the subtitle 'View and manage all your previous budget plans and recommendations'. The main section, titled 'Recent Recommendations', displays three budget cards:

- Jewelry Budget** (May 22, 2025, 12:06 PM): Total Budget ₹5,000, Remaining ₹800. Occasion: Birthday. With outfit image: Yes. [View Full Details](#)
- Party Planning Budget** (May 22, 2025, 12:05 PM): Total Budget ₹5,000, Remaining ₹0. Party Type: wedding. Guests: 3. Needs: Catering, Entertainment. [View Full Details](#)
- Home Interior Budget** (May 22, 2025, 12:02 PM): Total Budget ₹5,000, Remaining ₹500. Rooms: Living Room, Kitchen. Lights: 5, Fans: 4, Furniture: 2. [View Full Details](#)

Conclusion :

PocketSmart AI is an intelligent, AI-driven budget and lifestyle recommendation assistant that seamlessly integrates FastAPI, Gemini 1.5 Flash Pro, and a Jinja2-powered frontend. By processing user budgets, preferences, and images, the platform delivers context-aware suggestions across domains like home interiors, party planning, and jewelry via trusted services such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO. Designed with a focus on scalability and user experience, the system automates decision-making and streamlines everyday planning into a fast, personalized, and budget-friendly experience.